

《操作系统课程设计》A 题进度报告

项目题目：OS 内核实现

基础平台：xv6-riscv

课程名称	操作系统课程设计
报告性质	第 14 周进度报告
小组成员	涂凡琛、侯奕明、韩硕
当前选题	A 题：OS 内核实现
提交时间	2026 年 6 月

2026 年 6 月 7 日

目录

一、 项目基本情况	2
二、 当前进度报告	2
2.1 已完成的总体进度	2
2.2 已完成的具体工作	2
三、 xv6 当前已经实现的主要功能	3
3.1 系统启动	3
3.2 中断与异常处理	3
3.3 内存管理	4
3.4 进程管理	4
3.5 文件系统	5
3.6 用户程序加载与执行	5
四、 小组分工	5
4.1 涂凡琛：进程管理与处理机调度	6
4.2 侯奕明：接口、设备与软件交互	6
4.3 韩硕：内存管理与文件系统	6
4.4 共同完成内容	6
五、 技术方案	6
5.1 总体技术路线	6
5.2 针对 A 题要求的实现思路	7
5.3 后续扩展方向	8
六、 下一阶段计划	8
七、 总结	9

一、项目基本情况

本组选择《操作系统课程设计》项目制方案中的 A 题“OS 内核实现”。本项目以 `xv6-riscv` 为基础平台，围绕系统启动、中断与异常处理、内存管理、进程管理、文件系统以及用户程序加载与执行等核心模块展开，目标是在成功复现和理解现有系统的基础上，逐步完成课程要求中的各项功能，并在此之上继续扩展新的系统能力。

当前阶段属于项目中期的进度汇报阶段。本组的重点工作并不是从零重新搭建一套全新内核，而是在完成 `xv6` 运行环境复现的前提下，系统梳理其现有实现能力、分析其与课程 A 题要求之间的对应关系，并据此明确后续的完善方向、扩展方向和小组分工，为后续阶段的功能实现与汇报展示打下基础。

二、当前进度报告

2.1 已完成的总体进度

截至目前，本组已经完成 `xv6-riscv` 项目的获取、编译和运行验证，系统能够在 QEMU 环境中稳定启动，并成功进入 shell。该结果表明，当前平台的启动链路、基本内存系统、进程调度机制、文件系统及用户程序加载流程均已正常工作，本组已经具备一个可运行、可分析、可扩展的实验基础平台。

在完成系统复现之后，小组成员进一步结合课程设计 A 题的技术方案，对 `xv6` 的核心代码结构开展了分模块研读。当前阶段工作的重点已经从“把系统跑起来”转向“说明 `xv6` 已经实现了什么、这些实现与课程要求如何对应、后续还需要在哪些方面继续完善与扩展”。

2.2 已完成的具体工作

本组目前已经完成的工作主要包括以下几个方面：

1. 完成 `xv6-riscv` 的环境配置、源码获取、编译与运行。
2. 在 QEMU 中成功启动 `xv6`，验证系统可以进入 shell 并运行基础用户程序。
3. 对 `xv6` 的整体内核结构进行了初步划分，明确了系统启动、trap 机制、内存管理、进程管理、文件系统、设备与系统调用之间的关系。
4. 小组成员根据 A 题要求完成了分工，并已分别对各自负责模块展开代码阅读与结构分析。
5. 已明确后续技术路线：先基于 `xv6` 现有能力完成课程要求项的逐一对应与验证，再在此基础上继续补足、完善并扩展新的功能。

因此，当前阶段的成果可以概括为：**系统复现成功、整体框架明确、模块分工清晰、后续实现路线已确定。**

三、xv6 当前已经实现的主要功能

本节中的“已实现”主要指当前所采用的 `xv6-riscv` 基础平台已经具备的能力，以及本组已经完成的复现、运行验证和模块分析工作，而不是指本组已经从零独立实现了全部内核功能。下面按照课程设计 PDF 中 A 题的技术方案，对 xv6 当前已具备的核心能力进行归纳说明。

3.1 系统启动

在系统启动方面，xv6 已经实现了从 RISC-V 的 M 态切换到 S 态的过程，并完成了内核早期运行环境的建立。其启动流程包括入口汇编、栈初始化、特权级切换、页表启用前准备以及进入内核主函数 `main()`。随后由 `main()` 负责初始化物理页分配器、内核页表、进程表、trap 机制、文件系统以及第一个用户进程。

因此，从课程要求来看，xv6 已经具备以下能力：

- 能够从启动入口正确进入内核；
- 能够完成从 M 态到 S 态的切换；
- 能够完成栈和 C 语言运行环境准备；
- 能够稳定进入内核主函数并继续后续初始化。

这说明 xv6 已经满足 A 题中“系统启动”模块的基本功能要求，并且在当前实验环境中可以稳定复现。

3.2 中断与异常处理

在中断与异常处理方面，xv6 已经建立了较为完整的 trap 处理框架。系统通过 `stvec` 设置 trap 入口，区分用户态陷入与内核态陷入，并在 `trap.c` 中分别处理 `usertrap()` 与 `kerneltrap()` 两条主线。对于系统调用，xv6 支持用户态通过 `ecall` 进入内核，再由系统调用分发层转入具体服务逻辑。对于外部设备中断，系统当前可以处理控制台和虚拟磁盘等设备中断。对于时钟中断，系统能够通过定时器驱动调度循环并支持时间片让出。对于异常处理，当前版本中也包含了缺页异常对应的处理路径。

因此，xv6 当前已经具备：

- trap 入口设置与统一异常入口；
- 用户态到内核态的系统调用切换；

- 时钟中断与基础设备中断处理；
- 缺页异常处理基础路径。

这些内容与 A 题“中断与异常处理”的目标是对应的。

3.3 内存管理

在内存管理方面，xv6 已实现了基于 4KB 页的物理页管理与虚拟内存管理机制。系统中包含物理页分配器，用于管理可分配物理页；包含三级页表机制，用于完成虚拟地址到物理地址的映射；包含内核地址空间和用户地址空间的布局设计；并支持进程级页表创建、地址映射、解除映射和地址空间回收。此外，当前代码版本中还包含按需分页相关逻辑，即在特定场景下通过缺页异常触发物理页延迟分配。

从课程设计角度看，xv6 已经具备以下内存相关基础能力：

- 可管理页大小为 4KB 的物理内存页；
- 支持物理页分配与回收；
- 支持页表建立、地址映射和地址解除映射；
- 支持用户空间的代码段、数据段、堆栈等基础布局；
- 具备按需分配思路和实现基础。

因此，A 题中的“内存管理”要求在 xv6 中已经有较完整的参考实现基础。

3.4 进程管理

在进程管理方面，xv6 已经实现了进程控制块、进程状态转换、父子进程关系、上下文切换和基础调度机制。系统中支持进程创建、程序装载、进程退出和等待回收等流程，即 `fork/exec/wait/exit` 等核心机制均已存在。调度方面，xv6 当前采用基于时钟中断驱动的轮转式调度基础模型，可以支持多个进程交替运行。进程状态方面，系统已经区分 `UNUSED`、`USED`、`SLEEPING`、`RUNNABLE`、`RUNNING`、`ZOMBIE` 等状态。

因此，xv6 当前已经实现并可作为课程设计基础的内容包括：

- PCB 与进程基本信息管理；
- 进程创建、装载、退出与等待；
- 进程状态转换；
- 父子进程关系维护；
- 基础时间片轮转式调度。

这为后续按照 A 题要求进一步展示、分析和扩展调度策略提供了坚实基础。

3.5 文件系统

在文件系统方面, xv6 已经实现了一个较完整的教学型文件系统框架。系统包含缓冲区缓存、日志、inode、目录项、路径解析、文件对象、文件描述符管理等多层结构; 支持文件和目录的创建、打开、关闭、读写以及路径遍历; 并已经能够支撑 shell 和用户命令的正常运行。此外, xv6 配套提供了文件系统镜像构建工具, 可将用户程序和基础文件打包进镜像供系统启动后使用。

就 A 题要求而言, xv6 当前已经具备:

- 文件/目录创建、打开、关闭、读写能力;
- 文件描述符表管理;
- 路径解析;
- 文件系统镜像构建基础;
- shell 运行所需的基础文件系统支持。

后续工作将在这一基础上继续按照课程要求补充说明、验证与展示。

3.6 用户程序加载与执行

在用户程序加载与执行方面, xv6 已经实现了 ELF 程序加载器和基础用户态运行环境。系统启动后能够创建第一个用户进程, 随后执行 `/init`, 并进一步进入 shell。当前系统已经支持若干基础用户命令, 如 `ls`、`cat`、`echo` 等, 说明用户程序从磁盘镜像装入内存、建立用户态栈、返回用户态执行这一整条路径已经打通。

因此, xv6 当前已具备:

- ELF 格式用户程序加载;
- 用户态栈初始化;
- 第一个用户进程启动;
- shell 运行与基本命令执行。

这与 A 题“用户程序加载与执行”模块的目标高度对应。

四、 小组分工

本组共三名成员, 当前分工如下。

4.1 涂凡琛：进程管理与处理机调度

主要负责进程管理和处理机调度相关部分，重点包括进程控制块结构、进程状态机、调度器流程、上下文切换、fork/exec/wait/exit 等进程相关机制，以及进程与时间片调度之间的关系分析与后续扩展。

4.2 侯奕明：接口、设备与软件交互

主要负责系统调用接口、trap 路径、控制台/串口、设备中断以及软件与内核之间的交互逻辑。重点包括 ecall 到系统调用处理链路、时钟中断、设备中断、控制台输入输出、shell 与系统调用的接口关系等。

4.3 韩硕：内存管理与文件系统

主要负责物理页分配、页表管理、用户地址空间、按需分页相关机制，以及文件系统、inode、目录、缓冲区与日志等模块。重点包括 kalloc/vm 路线、虚拟内存映射、文件系统层次结构和文件读写链路分析。

4.4 共同完成内容

除上述分工外，整个小组共同承担以下工作：

- xv6 项目的整体框架搭建与环境配置；
- QEMU 运行验证与基础调试；
- 模块间联调与问题定位；
- 课程进度报告、设计文档和最终汇报材料整理。

这样的分工方式既保证了各成员有明确负责方向，也保证了整个系统在后续开发和汇报阶段能够形成统一协作。

五、 技术方案

5.1 总体技术路线

本组的技术路线不是脱离现有系统从零重新造一个内核，而是以 xv6-riscv 为基础实验平台，结合课程 A 题要求完成以下三步工作：

1. 成功复现并验证 xv6 现有功能；

2. 将 xv6 当前能力与 A 题要求逐项对应，找出可直接满足、需要补充展示、需要继续完善的部分；
3. 在此基础上逐步实现新的扩展功能，形成课程设计成果。

这种技术路线具有两方面优势：一方面可以立足一个已经可运行、可分析、可扩展的真实教学内核，降低系统级开发的起步难度；另一方面也能够使小组将精力集中在核心机制理解、需求对应、功能完善和创新扩展上，而不是耗费在重复搭建最基础内核框架上。

5.2 针对 A 题要求的实现思路

(1) 系统启动

后续将进一步结合 xv6 的启动链，整理从入口汇编、早期栈设置、M 态到 S 态切换、进入 `main()`、初始化内核子系统再到启动第一个用户进程的完整过程。该部分既作为实现分析内容，也作为汇报展示的重要主线。

(2) 中断与异常处理

后续将围绕 `trap.c`、`kernelvec.S`、`trampoline.S`、系统调用分发及设备中断处理过程，进一步梳理用户态到内核态的切换路径，并结合时钟中断与缺页异常的处理机制形成完整说明。

(3) 内存管理

后续将以 `kalloc.c` 和 `vm.c` 为基础，对物理页分配、页表构造、地址映射、用户空间布局和按需分页机制进行梳理，并进一步验证它们与课程要求中的 4KB 页、页表映射、内存释放复用等目标的对应关系。

(4) 进程管理

后续将围绕 `proc.c`、`swtch.S`、调度器、上下文切换和进程生命周期进行说明，并在现有轮转调度基础上，考虑继续完善与扩展调度展示方式。

(5) 文件系统

后续将围绕缓冲区、日志、inode、目录与路径解析结构，整理文件系统的层次设计，并结合 shell 中实际文件操作过程来说明 xv6 如何支撑用户命令运行。

(6) 用户程序加载与执行

后续将重点分析 ELF 加载、用户页表建立、trap 返回用户态、`/init` 启动和 shell 执行流程，形成从内核到用户程序的完整执行链条说明。

5.3 后续扩展方向

在完成 A 题基础功能要求的基础上，本组计划继续向以下三个方向扩展。

1. 系统增强

- 继续完善进程调度与执行链分析；
- 深入利用当前版本中的 lazy allocation 机制；
- 完善 shell、管道、命令执行等系统层面行为展示。

2. 文件系统增强

- 进一步强化路径、重定向、文件操作和文件系统镜像相关展示；
- 结合实验需要补充更完整的文件系统行为验证。

3. 用户程序增强

- 在现有命令基础上继续扩展诸如 ps、kill、exec 等命令；
- 增加更多用户程序和演示样例，用于展示内核功能。

总的来说，本组后续的技术方案是：先完成课程要求的“基础达成”，再在已有 xv6 平台之上做“功能增强与扩展创新”。

六、 下一阶段计划

下一阶段，本组将重点推进以下工作：

1. 按照 A 题六大模块继续完成代码级分析与实验验证；
2. 将 xv6 已实现能力与课程要求逐项对应，明确哪些内容已经具备、哪些需要补充展示、哪些需要进一步完善；
3. 在现有基础上继续完成新增功能设计与实现；
4. 补充系统运行结果、流程图、截图、模块说明和展示材料；
5. 为后续答辩和汇报准备更加完整的技术文档与演示内容。

七、 总结

当前阶段，本组已经成功完成 xv6-riscv 的复现和运行验证，并在此基础上开展了较系统的代码阅读与模块梳理工作。通过对系统启动、中断异常、内存管理、进程管理、文件系统和用户程序加载等模块的分析，可以确认 xv6 已经为 A 题所要求的内核功能提供了较完整的基础平台。

后续，本组将继续在这一基础上，把“可运行的 xv6”进一步转化为“符合课程设计要求并带有扩展能力的实验项目成果”，逐步完成 A 题要求的全部功能，并在系统、文件和用户程序三个方向继续增加新功能。